Superpixels SLIC : de l'Image Couleur au Cube Hyperspectral

Réimplémentation de l'algorithme, évaluation sur la base de Berkeley et extension aux données hyperspectrales et volumiques

Rapport de Projet – Traitement d'Images

Télécom Paris, 2025, DIONNET ELIOTT

Avec la contribution de LE NILIOT ENGUERRAND

Article étudié : R. Achanta *et al.*, *SLIC Superpixels Compared to State-of-the-Art Superpixel Methods*, IEEE TPAMI, 2012

2 novembre 2026

Résumé

L'algorithme SLIC (*Simple Linear Iterative Clustering*) découpe une image en superpixels par un partitionnement en k-moyennes restreint à un voisinage local, dans un espace qui réunit la couleur et la position. Ce rapport en présente une réimplémentation complète en Python, de l'initialisation sur le minimum de gradient jusqu'au rattachement des régions isolées, puis l'évalue sur une image de la base de segmentation de Berkeley face à la version de scikit-image. Avec $K = 200$, notre implémentation atteint une erreur de sous-segmentation moyenne de 0,094 contre 0,115 pour scikit-image, au prix d'un temps de calcul plus de vingt fois plus long. La méthode est ensuite étendue aux images hyperspectrales, avec un gradient et une distance calculés sur toutes les bandes (Salinas, 204 bandes ; Pavia University, 103 bandes), puis aux volumes en trois dimensions sous forme de supervoxels. Le fil conducteur des résultats est le paramètre de compacité m : sa plage utile dépend entièrement de l'échelle des valeurs du signal. Sur Salinas en radiance brute, m varie d'un facteur 10 sans effet mesurable, tandis que sur des volumes normalisés entre 0 et 1, passer de $m = 10$ à $m = 0{,}5$ ramène l'erreur de sous-segmentation de la sphère synthétique de 0,150 à 0,000.

1 Introduction et Contexte

Un superpixel est un groupe de pixels voisins et semblables, que l'on manipule ensuite comme une seule unité. Remplacer les centaines de milliers de pixels d'une image par quelques centaines de régions qui respectent ses contours réduit fortement le coût des traitements suivants (segmentation, classification, suivi d'objets) sans effacer l'information de forme. Une bonne méthode de superpixels doit donc coller aux frontières réelles des objets, produire des régions de taille comparable et rester rapide.

Achanta *et al.* [1] ont proposé SLIC, qui reformule le problème comme un partitionnement en k-moyennes dans un espace à cinq dimensions $[l, a, b, x, y]$, avec une seule modification décisive : chaque centre ne cherche ses pixels que dans une fenêtre de taille $2S \times 2S$ autour de lui. Le coût par itération devient linéaire en le nombre de pixels N et ne dépend plus du nombre de superpixels K. L'article montre que cette méthode très simple égale ou dépasse les méthodes de l'époque, comme la segmentation par graphe de Felzenszwalb et Huttenlocher [4], sur la base de segmentation de Berkeley [2].

Ce projet poursuit trois objectifs. Le premier est de réimplémenter SLIC sans recourir à une bibliothèque de superpixels, pour en comprendre chaque étape. Le second est de mesurer la qualité de la segmentation obtenue avec les métriques de l'article et d'étudier l'influence de ses deux paramètres, K et m. Le troisième est d'étendre la méthode au-delà de l'image couleur : aux images hyperspectrales, où chaque pixel porte un spectre de plus de cent bandes, puis aux volumes en trois dimensions.

Le rapport suit cet ordre. La section 2 décrit l'algorithme et la section 3 notre implémentation. La section 4 présente le protocole d'évaluation, dont les résultats sur image couleur figurent en section 5. Les sections 6 et 7 traitent des extensions hyperspectrale et volumique, avant une discussion des limites en section 8.

2 L'Algorithme SLIC

2.1 Une Distance Mixte Couleur et Position

SLIC travaille dans l'espace CIELAB, choisi parce que les distances euclidiennes y suivent à peu près les écarts de couleur perçus. Un pixel i est décrit par $[l_i, a_i, b_i, x_i, y_i]$. La distance entre un centre k et un pixel i combine une distance colorimétrique et une distance spatiale :

$$d_c = \sqrt{(l_i - l_k)^2 + (a_i - a_k)^2 + (b_i - b_k)^2}, \quad (1)$$

$$d_s = \sqrt{(x_i - x_k)^2 + (y_i - y_k)^2}, \quad (2)$$

$$D = \sqrt{d_c^2 + \left(\frac{d_s}{S}\right)^2 m^2}. \quad (3)$$

La distance spatiale est normalisée par le pas de grille S, qui fixe la taille attendue d'un superpixel, et le paramètre

de compacité m règle le poids de la position face à la couleur. Un m grand produit des régions presque carrées et régulières ; un m petit laisse la couleur dominer et donne des régions qui épousent mieux les contours, mais plus irrégulières. L'article recommande $m \in [1, 40]$ pour des couleurs CIELAB.

2.2 Les Étapes de l'Algorithme

Pour une image de N pixels et un nombre cible K de superpixels, l'algorithme se déroule ainsi.

1. **Grille initiale.** Les centres sont posés sur une grille régulière de pas
$$S = \sqrt{N/K}, \tag{4}$$
de sorte que chaque superpixel couvre en moyenne S^2 pixels.
2. **Déplacement vers le gradient minimal.** Chaque centre est déplacé vers le pixel de plus faible gradient dans son voisinage 3×3, pour éviter de démarrer sur un contour ou sur un pixel bruité.
3. **Affectation locale.** Chaque centre compare la distance D de l'équation (3) aux pixels de la fenêtre $2S \times 2S$ qui l'entoure. Un pixel rejoint le centre le plus proche parmi ceux qui l'ont examiné.
4. **Mise à jour.** Chaque centre devient la moyenne $[l, a, b, x, y]$ des pixels qui lui sont affectés.
5. **Itération.** Les étapes 3 et 4 sont répétées jusqu'à ce que les centres ne bougent presque plus. L'article indique que dix itérations suffisent pour la plupart des images.
6. **Connexité.** Rien dans les k-moyennes ne garantit qu'un superpixel soit d'un seul tenant. Les fragments isolés sont rattachés à un superpixel voisin.

Comme chaque pixel n'est examiné que par les quelques centres dont la fenêtre le recouvre, l'étape d'affectation coûte $O(N)$ par itération, quel que soit K. C'est la propriété qui fait la rapidité de SLIC.

3 Notre Implémentation

L'ensemble est écrit en Python avec NumPy. OpenCV ne sert qu'à la conversion vers l'espace Lab et aux filtres de Sobel ; aucune fonction de superpixels n'est utilisée. Les choix d'implémentation sont les suivants.

Couleur et gradient. L'image RGB est convertie en Lab par OpenCV sur 8 bits, puis le gradient est calculé par des filtres de Sobel 3×3 sur chacun des trois canaux :
$$G = \sqrt{\textstyle\sum_{c \in \{L,a,b\}} \left(\partial_x I_c\right)^2 + \left(\partial_y I_c\right)^2}. \tag{5}$$
L'article utilise des différences centrées ; le filtre de Sobel ajoute un léger lissage, ce qui stabilise le choix des centres sur les zones bruitées.

Itérations et arrêt. Plutôt qu'un nombre fixe d'itérations, nous arrêtons la boucle quand le déplacement spatial moyen des centres passe sous 0,5 pixel, avec un plafond de 20 itérations.

Connexité. Un parcours en profondeur sur le voisinage à quatre pixels identifie chaque composante connexe. Une composante de moins de $S^2/4$ pixels est rattachée au superpixel avec lequel elle partage le plus de frontière, et les étiquettes sont renumérotées au passage.

Le nombre final de superpixels. Le pas S est arrondi à l'entier inférieur, et la grille est posée à partir de $S/2$ jusqu'au bord de l'image. Le nombre final de superpixels K_f diffère donc de K et évolue par paliers : plusieurs valeurs de K donnent le même pas S.

4 Protocole d'Évaluation

4.1 Données

La partie en couleur utilise l'image 3096 de la base de segmentation de Berkeley [2], de 481×321 pixels, qui montre un avion sur un ciel nuageux. Ses frontières mêlent un objet net et des transitions douces, ce qui en fait un bon test de l'adhérence aux contours. Nous utilisons les cinq segmentations humaines fournies avec la version BSDS500 de la base [3], qui comptent de 3 à 11 régions selon l'annotateur, et nous moyennons les métriques sur les cinq.

4.2 Métriques

Erreur de sous-segmentation. Elle mesure à quel point les superpixels débordent des régions de la vérité terrain. Pour des régions de référence $g_1, \ldots, g_M$ et des superpixels s_j, l'article la définit par
$$U = \frac{1}{N} \left[\sum_{i=1}^{M} \sum_{s_j \,:\, |s_j \cap g_i| > B} |s_j| \;-\; N \right], \tag{6}$$
où B vaut 5 % de la taille $|s_j|$ et absorbe les petits défauts d'alignement. Un superpixel contenu dans une seule région compte une fois et un superpixel à cheval sur deux régions compte deux fois. U vaut donc 0 pour un découpage parfait et augmente avec les débordements.

Rappel des contours. Le *boundary recall* est la fraction des pixels de contour de la vérité terrain qui se trouvent à moins de 2 pixels d'une frontière de superpixel. Il vaut 1 quand tous les vrais contours sont retrouvés.

Alignement au gradient. Nous ajoutons un indicateur qui ne demande pas de vérité terrain, le rapport entre le gradient moyen sur les frontières des superpixels et le gradient moyen de toute l'image :
$$\mathrm{GA} = \frac{\mathrm{moy}_{\partial \mathcal{S}}\, G}{\mathrm{moy}\, G}. \tag{7}$$
Une valeur supérieure à 1 signifie que les frontières passent préférentiellement sur des zones de fort gradient, donc sur des bords.

Référence. Nous comparons nos résultats à la fonction `slic` de scikit-image [5], une implémentation compilée de la même méthode, appelée avec les mêmes K et m.

Exécution. Les expériences ont été exécutées avec le code du projet sur une machine à deux cœurs. Les temps n'ont de sens qu'en comparaison les uns avec les autres.

5 Résultats sur Image Couleur

5.1 Comparaison Qualitative

La figure 1 montre les superpixels obtenus pour $K = 200$. Pour $m = 25$, notre implémentation et celle de scikit-image produisent des régions compactes de taille voisine, et toutes deux détourent la silhouette de l'avion, ailes et fuselage compris. Notre version produit 216 superpixels contre 184 pour scikit-image. Ce dernier élimine par défaut les régions de moins de la moitié de la taille attendue, alors que notre seuil est d'un quart.

Le paramètre m change nettement l'allure du découpage. Avec $m = 5$, les frontières suivent les variations de gris des nuages et les régions prennent des formes très irrégulières : le coefficient de variation de leur taille passe de 0,18 pour $m = 20$ à 0,33 pour $m = 5$.

5.2 Influence de la Compacité

Sur cette image, l'erreur de sous-segmentation reste comprise entre 0,086 et 0,094 quand m va de 5 à 20 (figure 2a). La compacité modifie l'allure des régions bien plus qu'elle ne modifie leur débordement : la plupart des annotateurs ne découpent le ciel qu'en très peu de régions, si bien que suivre le contour des nuages ne rapporte presque rien en U. À $K = 200$ et $m = 25$, notre implémentation obtient $U = 0{,}094$ contre 0,115 pour scikit-image. L'écart se retrouve chez chacun des cinq annotateurs pris séparément, de 0,028 à 0,226 pour notre version.

5.3 Influence du Nombre de Superpixels

L'erreur décroît régulièrement avec le nombre de superpixels (figure 2b), de 0,128 pour $K_f = 96$ à 0,052 pour $K_f = 2386$. Des superpixels plus petits débordent moins, ce qui est le comportement attendu. Les deux implémentations suivent la même tendance, et notre version garde une erreur plus faible à nombre de superpixels comparable. Les paliers de K_f décrits en section 3 sont visibles : $K = 1100$ et $K = 1200$ donnent tous deux $K_f = 1274$.

5.4 Temps de Calcul

C'est le point faible de notre version (figure 2c) : 2,1 s pour $K = 100$ et 44,2 s pour $K = 2000$, contre 0,12 à 0,23 s pour scikit-image. Une partie de l'écart vient de Python lui-même, puisque le contrôle de connexité parcourt les pixels un par un. L'autre partie vient de la mise à jour des centres : pour chaque centre, nous construisons un masque sur l'image entière, ce qui coûte $O(NK)$ par itération au lieu de $O(N)$. C'est ce terme qui fait croître le temps avec K. Une accumulation des sommes en un seul passage sur l'image, avec `np.bincount`, rendrait cette étape indépendante de K.

TABLE 1 : Salinas, métriques selon m pour $K = 200$ et selon K pour $m = 20$. U est calculée avec $B = 5\,\%$, BR désigne le rappel des contours.

K	m	K_f	U	BR	GA
200	5	198	0,523	0,912	2,01
200	10	198	0,480	0,915	2,00
200	20	198	0,513	0,909	2,02
200	30	198	0,505	0,908	2,02
200	40	198	0,513	0,912	2,02
200	50	198	0,486	0,911	2,01
50	20	55	0,814	0,780	2,65
100	20	112	0,602	0,866	2,10
200	20	198	0,513	0,909	2,02
500	20	549	0,362	0,964	1,62
1000	20	1107	0,322	0,979	1,45
2000	20	2228	0,247	0,994	1,29

6 Extension Hyperspectrale

6.1 Adaptation de la Méthode

Une image hyperspectrale est un cube $h \times w \times B$ dont chaque pixel porte un spectre de B bandes. L'espace Lab n'a plus de sens, et nous travaillons directement sur les spectres :

- la distance colorimétrique devient la norme euclidienne entre spectres, $d_c = \|\mathbf{x}_i - \mathbf{x}_k\|_2$ sur les B bandes ;
- le gradient de l'équation (5) somme les dérivées de Sobel des B bandes ; pour l'alignement au gradient, il est calculé sur le cube normalisé entre 0 et 1 ;
- les centres stockent un spectre moyen et une position, soit $B + 2$ composantes ;
- l'initialisation, l'affectation locale, le critère d'arrêt et le contrôle de connexité sont inchangés.

L'affectation reste en $O(N)$ par itération, mais chaque comparaison coûte désormais $O(B)$.

6.2 Données

Salinas est une scène agricole de Californie acquise par le capteur aéroporté AVIRIS [6]. Elle mesure 512×217 pixels et compte 204 bandes une fois retirées les bandes d'absorption de l'eau. Sa vérité terrain distingue 16 classes de cultures, et les pixels non étiquetés forment une région de fond. Pavia University est une scène urbaine acquise par le capteur ROSIS [6], de 610×340 pixels et 103 bandes, avec 9 classes étiquetées.

6.3 Résultats sur Salinas

La figure 3 montre le découpage pour $K = 300$ et $m = 10$. Les grandes parcelles homogènes sont découpées en régions qui suivent les limites de champs, avec un rappel des contours de 0,927 et un alignement au gradient de 1,87. En revanche, les frontières sont très découpées : à l'intérieur d'une même parcelle, les superpixels forment des motifs irréguliers.

Le tableau 1 explique pourquoi. Quand m passe de 5 à 50, les trois métriques restent quasi constantes : U

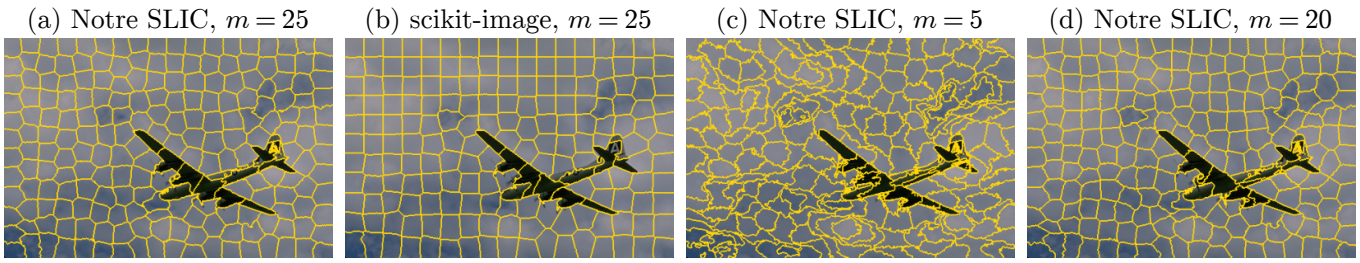


FIGURE 1 : Superpixels sur l'image BSDS 3096 avec $K = 200$. (a) et (b) : notre implémentation et scikit-image pour $m = 25$. (c) et (d) : notre implémentation pour $m = 5$ et $m = 20$. Avec $m = 5$, la couleur domine et les frontières suivent les nuages, au prix de régions très irrégulières.

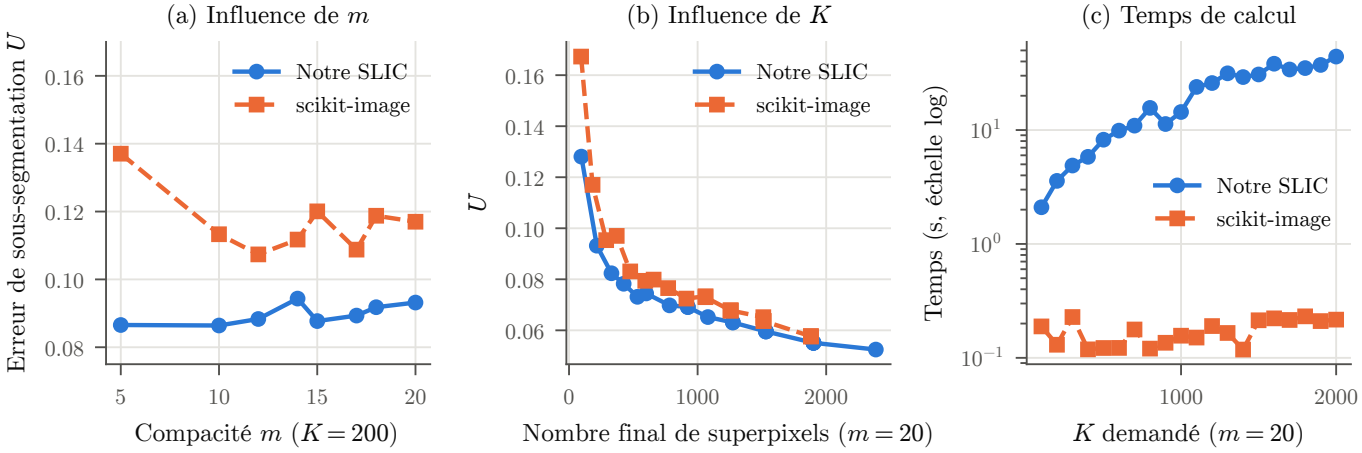


FIGURE 2 : Image BSDS 3096, erreur moyennée sur les cinq annotateurs. (a) Erreur de sous-segmentation selon la compacité m pour $K = 200$. (b) Erreur selon le nombre final de superpixels pour $m = 20$. (c) Temps de calcul selon K, en échelle logarithmique.

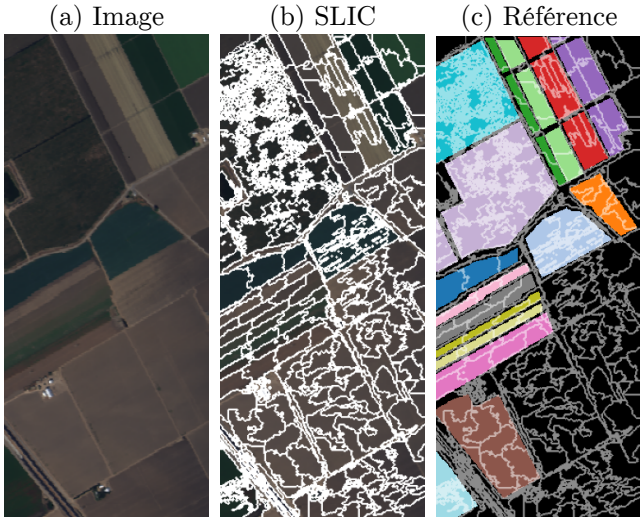


FIGURE 3 : Salinas. (a) Composition colorée des bandes 29, 19 et 9. (b) Superpixels pour $K = 300$ et $m = 10$. (c) Vérité terrain, avec les frontières des superpixels en surimpression.

entre 0,480 et 0,523, BR entre 0,908 et 0,915. Le terme spatial ne pèse presque rien. Le cube est en effet traité en radiance brute, avec des valeurs allant jusqu'à 9 207. La distance spectrale médiane entre deux pixels décalés d'un demi-pas de grille dans chaque direction vaut 1 650, alors que le terme spatial $m\, d_s/S$ ne dépasse pas $\sqrt{2}\, m$ dans la fenêtre de recherche, soit environ 70 pour $m = 50$. Il faudrait des valeurs de m de l'ordre du millier pour retrouver l'équilibre prévu par l'article. La plage $[1, 40]$ n'a de sens que pour l'échelle des couleurs CIELAB.

Le nombre de superpixels agit en revanche comme attendu. De $K = 50$ à $K = 2000$, l'erreur U chute de 0,814 à 0,247 et le rappel monte de 0,780 à 0,994. L'alignement au gradient baisse, de 2,65 à 1,29 : avec des régions plus petites, une part croissante des frontières traverse forcément des zones homogènes. Le temps de calcul reste entre 12 et 25 s sur toute la plage de K, sans croître avec lui : l'affectation, qui compare des spectres de 204 bandes, coûte autant quelle que soit la valeur de K.

6.4 Homogénéité Spectrale sur Pavia

Sur Pavia, nous avons étudié ce que contient un superpixel plutôt que la position de ses frontières. Le cube est normalisé entre 0 et 1 et segmenté avec $K = 200$ et $m = 1$ par une variante plus légère de l'implémentation, qui effectue dix itérations sans contrôle de connexité. Elle formule la distance $\sqrt{(d_c/m)^2 + (d_s/S)^2}$, qui vaut l'équation (3) divisée par m et classe donc les centres de la même façon. On obtient 209 superpixels, et l'absence de contrôle de connexité se voit nettement : un superpixel est éclaté en 15 morceaux disjoints en médiane, ce qui illustre le rôle de la dernière étape de l'algorithme. Pour chaque superpixel, nous calculons l'écart-type de chaque bande sur ses pixels, puis la moyenne sur les 103 bandes, notée $\bar{\sigma}$.

La figure 4 montre que les superpixels suivent les bâti-

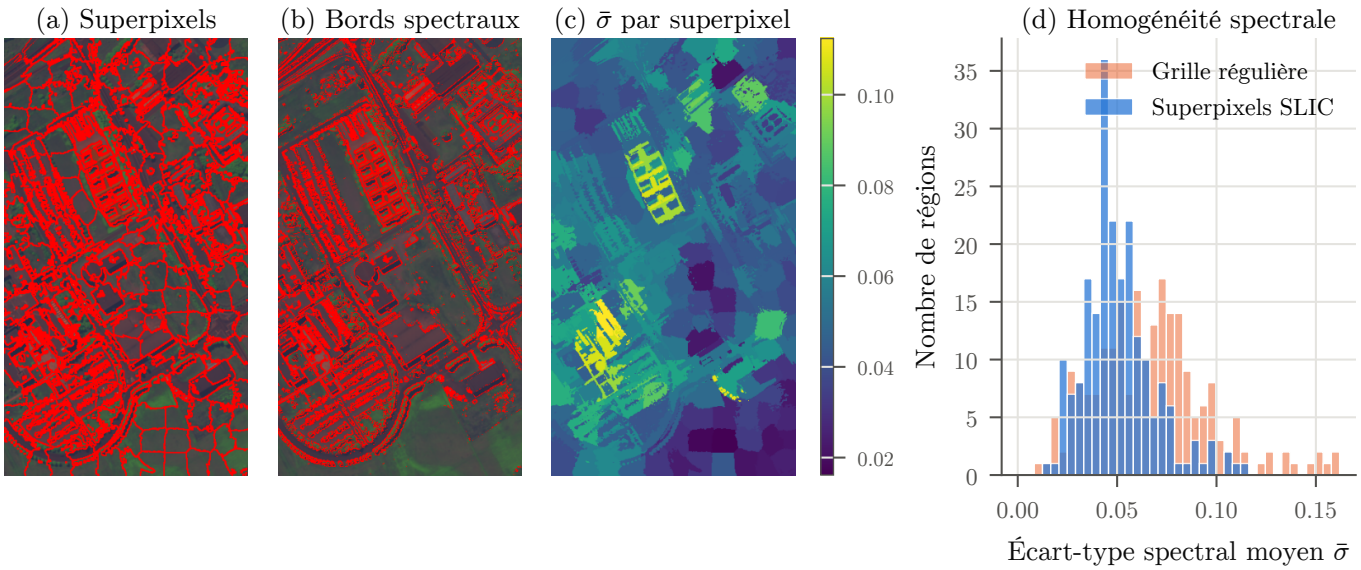


FIGURE 4 : Pavia University, $K = 200$ et $m = 1$ sur le cube normalisé. (a) Frontières des superpixels sur une composition colorée des trois premières composantes principales. (b) Bords spectraux, obtenus en seuillant le gradient spectral. (c) Écart-type spectral moyen de chaque superpixel. (d) Distribution de cet écart-type pour les superpixels et pour une grille régulière de blocs de même taille.

ments, les routes et les zones de végétation repérées par les contours spectraux. Leur écart-type spectral médian vaut 0,0468, contre 0,0663 pour une grille régulière de blocs de même taille moyenne. Rapportée aux 9 classes de la vérité terrain, la pureté moyenne des superpixels, c'est-à-dire la part de leurs pixels étiquetés qui appartient à la classe majoritaire, atteint 93 % contre 85 % pour la grille. La comparaison avantage un peu SLIC, puisque ses superpixels fragmentés peuvent réunir des pixels semblables qui ne se touchent pas. Les trois premières composantes principales portent 98,9 % de la variance du cube, ce qui confirme que l'information spectrale est très redondante et qu'une réduction de dimension avant SLIC serait possible.

7 Extension aux Volumes

7.1 Supervoxels

L'article évoque l'extension de SLIC aux vidéos et aux volumes. Les changements sont directs :

- le pas de grille devient $S = \sqrt[3]{N/K}$ et les centres forment une grille cubique ;
- le déplacement initial cherche le gradient minimal dans un voisinage $3 \times 3 \times 3$;
- la fenêtre de recherche devient un cube $2S \times 2S \times 2S$ et d_s une distance en trois dimensions ;
- le contrôle de connexité utilise le voisinage à six voxels et un seuil de $S^3/4$ voxels.

Les métriques s'étendent de la même façon : les contours deviennent des surfaces et le rappel se calcule avec une tolérance d'un voxel.

7.2 Données et Résultats

Nous testons les supervoxels sur deux volumes normalisés entre 0 et 1. Le premier est synthétique : une boule d'intensité 0,8 dans un fond à 0,2, sur une grille de $30 \times 100 \times 100$ voxels, avec un bruit gaussien d'écart-type 0,05, le tout renormalisé entre 0 et 1. Le second est une tranche de 10 coupes axiales du gabarit cérébral T1 MNI152 [7], de $10 \times 233 \times 197$ voxels, dont les cartes de probabilité de substance grise et blanche, seuillées à 0,5, servent de référence.

TABLE 2 : Supervoxels, $K = 200$. Pour la sphère, la vérité terrain est la boule et le fond. Pour l'IRM, elle sépare substance grise, substance blanche et reste du volume.

Volume	m	U	BR	GA
Sphère	0,50	0,000	1,000	1,22
Sphère	1	0,105	0,932	1,11
Sphère	2	0,143	0,715	1,01
Sphère	5	0,150	0,667	0,99
Sphère	10	0,150	0,662	0,99
Sphère	20	0,150	0,661	0,99
IRM	0,25	0,350	0,781	1,93
IRM	0,50	0,388	0,629	1,76
IRM	1	0,419	0,564	1,56
IRM	2	0,421	0,562	1,17
IRM	10	0,442	0,565	1,04

Le même phénomène que sur Salinas apparaît, dans l'autre sens. L'écart d'intensité entre deux tissus ne dépasse pas 1, tandis que le terme spatial $m\, d_s/S$ atteint $\sqrt{3}\, m$ dans la fenêtre de recherche. Avec $m = 10$, la position domine et les supervoxels restent des cubes alignés sur la grille (figure 5a et c). Sur la sphère, le rappel des contours ne vaut que 0,662. En abaissant m à 0,5, les supervoxels suivent exactement la boule : U tombe à 0,000 et le rappel atteint 1,000 (tableau 2). Sur l'IRM, passer de $m = 10$ à $m = 0{,}25$ fait monter le rappel des contours de 0,565 à 0,781 et l'alignement au gradient de 1,04 à 1,93. Les supervoxels épousent alors le contour du

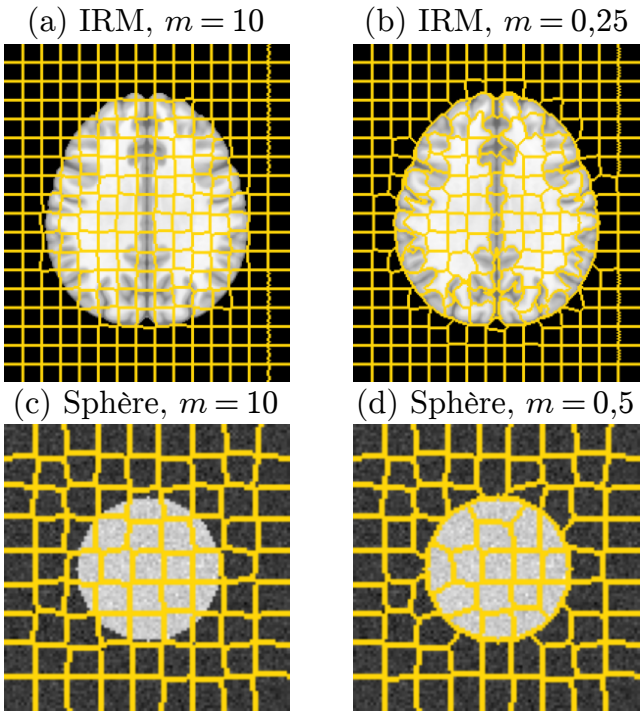


FIGURE 5 : Coupes centrales des supervoxels pour $K = 200$. (a) et (b) : tranche IRM pondérée T1. (c) et (d) : sphère synthétique bruitée. Avec $m = 10$, les supervoxels restent des cubes ; avec un m adapté à l'échelle $[0, 1]$, ils suivent les contours.

cerveau et ses structures internes (figure 5b).

À $m = 10$ fixé sur la sphère, augmenter K de 50 à 800 fait passer U de 0,256 à 0,107 et le rappel de 0,514 à 0,931 : plus de supervoxels compensent en partie une compacité mal réglée, mais pour un coût en mémoire et en temps bien supérieur.

8 Discussion et Limites

Le réglage de m. C'est l'enseignement principal du projet. L'équation (3) compare une distance dans l'espace des valeurs à une distance spatiale normalisée, et m n'a de sens que rapporté à l'échelle de ces valeurs. La plage de l'article vaut pour des couleurs CIELAB. Elle est déjà décalée par la conversion Lab d'OpenCV sur 8 bits, qui étire la luminance de $[0, 100]$ à $[0, 255]$, ce qui explique en partie le faible effet de m sur l'image BSDS. Elle devient trop faible d'un ou deux ordres de grandeur pour une radiance hyperspectrale brute, et trop forte pour des volumes normalisés. Une règle simple consiste à normaliser les données, puis à choisir m à partir de la distance spectrale typique entre pixels voisins, mesurée sur l'image elle-même. La variante SLICO de l'article, qui adapte m à chaque superpixel, va dans ce sens.

La distance spectrale. La norme euclidienne sur des radiances confond une différence de matériau avec une simple différence d'éclairement, qui multiplie tout le spectre par un même facteur. La distance angulaire du *Spectral Angle Mapper* [8] y est insensible et serait plus pertinente pour des scènes ombragées comme Pavia. Réduire le cube à ses premières composantes principales avant SLIC diviserait aussi le coût de chaque comparaison.

Le coût de calcul. Notre implémentation reste lente, à cause des boucles Python du contrôle de connexité et de la mise à jour en $O(NK)$ des centres. Ces deux étapes se vectorisent, ou se compilent avec Numba, sans changer l'algorithme.

L'évaluation. Les résultats en couleur portent sur une seule image, et la métrique U dépend du découpage choisi par les annotateurs. Sur l'image 3096, le ciel n'est découpé qu'en très peu de régions par la plupart des annotateurs, ce qui rend U peu sensible à m. Une évaluation sur l'ensemble de la base de Berkeley, comme dans l'article, donnerait des courbes plus représentatives. En trois dimensions, la référence de l'IRM est elle-même approximative puisqu'elle provient de cartes de probabilité seuillées.

9 Conclusion

Ce projet a réimplémenté SLIC en entier, de l'initialisation sur le minimum de gradient jusqu'au contrôle de connexité. Sur l'image BSDS 3096, notre version obtient une erreur de sous-segmentation légèrement plus faible que celle de scikit-image à paramètres égaux, avec des régions de qualité visuelle comparable, mais un temps de calcul bien plus long.

Les extensions montrent que la méthode se généralise facilement. Le passage aux spectres et aux volumes ne change ni la structure de l'algorithme ni sa complexité linéaire. Elles montrent aussi que sa simplicité a un prix : tout l'équilibre entre adhérence aux contours et régularité repose sur un paramètre m dont la valeur pertinente dépend de l'échelle des données. Bien réglé, SLIC découpe proprement une scène hyperspectrale de 204 bandes ou une IRM en trois dimensions. Mal réglé, il ignore la position, comme sur Salinas en radiance brute, ou ignore le signal, comme sur les volumes normalisés avec $m = 10$.

Références

[1] R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua et S. Süsstrunk, « SLIC Superpixels Compared to State-of-the-Art Superpixel Methods », *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 34, n° 11, pp. 2274–2282, 2012.

[2] D. Martin, C. Fowlkes, D. Tal et J. Malik, « A Database of Human Segmented Natural Images and its Application to Evaluating Segmentation Algorithms and Measuring Ecological Statistics », *ICCV*, IEEE, 2001.

[3] P. Arbeláez, M. Maire, C. Fowlkes et J. Malik, « Contour Detection and Hierarchical Image Segmentation », *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, n° 5, pp. 898–916, 2011.

[4] P. Felzenszwalb et D. Huttenlocher, « Efficient Graph-Based Image Segmentation », *International Journal of Computer Vision*, vol. 59, n° 2, pp. 167–181, 2004.

[5] S. van der Walt *et al.*, « scikit-image : Image Processing in Python », *PeerJ*, vol. 2, e453, 2014.

[6] Grupo de Inteligencia Computacional, Universidad del País Vasco, « Hyperspectral Remote Sensing Scenes », jeux de données Salinas et Pavia University.

[7] V. Fonov, A. C. Evans, K. Botteron, C. R. Almli, R. C. McKinstry et D. L. Collins, « Unbiased Average Age-Appropriate Atlases for Pediatric Studies », *NeuroImage*, vol. 54, n° 1, pp. 313–327, 2011.

[8] F. A. Kruse *et al.*, « The Spectral Image Processing System (SIPS) : Interactive Visualization and Analysis of Imaging Spectrometer Data », *Remote Sensing of Environment*, vol. 44, n° 2–3, pp. 145–163, 1993.